Que1: What is a conditional statement in programming?

Ans: A conditional statement is a programming construct that allows a program to make decisions based on whether a condition is true or false.

Explanation

Conditional statements help control the flow of a program. They execute different blocks of code depending on the result of a condition

Que2: What does an if statement do?

Ans: An if statement checks a condition and executes a block of code only if the condition is true.

Example:

```
if(age >= 18)
{
   cout << "Eligible to vote";
}
```

Explanation

The code inside the if block runs only when the specified condition evaluates to true.

Que3: What is the purpose of an else block?

Ans: The else block is used to execute a block of code when the condition in the if statement is false.

Explanation

It provides an alternative action when the specified condition is not satisfied.

Que4: When is an else block executed?

Ans: An else block is executed when the condition associated with the if statement evaluates to false.

Explanation

The program enters the else block only if the if condition fails.

Que5: What is a Boolean expression?

Ans: A Boolean expression is an expression that evaluates to either true or false.

Example:

Age >= 18;

Explanation

Boolean expressions are commonly used in conditional statements to make decisions.

Que6: Write the syntax of a simple if statement.

Ans: if(condition)

{

  // statements

}

Explanation

The statements inside the braces execute only when the condition is true.

Que7: What operator is commonly used to test equality?

Ans: The equality operator == is used to test whether two values are equal.

Example:

If(a==b){

return a

}

Explanation

The operator returns true when both operands have the same value; otherwise, it returns false.

Que8: What does the modulus (%) operator return?

Ans: The modulus (%) operator returns the remainder after division.

For example: 10%3 = 1;

Explanation

It is commonly used to check divisibility and determine whether numbers are even or odd.

Que9: How can you check if a number is even?

Ans: if(num % 2 == 0) it means it's a Even no.

Explanation

If a number leaves a remainder of 0 when divided by 2, it is an even number.

Que10: How can you check if a number is odd?

Ans: if(num % 2 != 0)

Explanation

If a number leaves a remainder other than 0 when divided by 2, it is an odd number.

Que11: What is the purpose of else-if?

Ans: The else-if statement is used to check multiple conditions one after another. If the first condition is false, the program checks the next condition.

Example:

```
if(marks >= 90)
{
   cout << "Grade A";
}
```

```
else if(marks >= 75)
{
   cout << "Grade B";
}
```

Explanation

The else-if statement helps in making decisions among multiple possible conditions.

Que12: What is a nested if statement?

Ans: A nested if statement is an if statement placed inside another if statement.

Example:

```
if(age >= 18)
{
   if(citizen == true)
   {
      cout << "Eligible to vote";
   }
}
```

Explanation

Nested if statements are used when one condition depends on another condition being true.

Que13: Can an if statement exist without an else?

Ans: Yes, an if statement can exist without an else statement.

Example:

```
if(age >= 18)
{
   cout << "Adult";
}
```

Explanation

The else block is optional. If the condition is false and no else exists, nothing happens.

Que14: What is the output when a false condition is tested in an if statement without else?

Ans: No output is produced and the statements inside the if block are skipped.

Example:

```
if(5 > 10)
{
   cout << "Hello";
}
```

Explanation

Since the condition is false, the compiler ignores the code inside the if block and continues with the next statement.

Que15: What is the role of braces {} in C/C++ conditionals?

Ans: Braces {} are used to group multiple statements into a single block.

Example:

```
if(age >= 18)
{
   cout << "Eligible";
   cout << " to vote";
}
```

Explanation

Braces clearly define which statements belong to the conditional block and improve code readability.

Que16: What does > mean?

Ans: The > operator means **greater than**.

Example:

If(a > b){

cout<<a;

}

Explanation

It checks whether the value on the left side is greater than the value on the right side. It returns true if the condition is satisfied.

Que17: What does < mean?

Ans: The < operator means **less than**.

Example:

If(a < b){

cout<<b;

}

Explanation

It checks whether the value on the left side is smaller than the value on the right side. It returns true if the condition is correct.

Que18: What does >= mean?

Ans: The >= operator means **greater than or equal to**.

Example:

If(age >= 18){

cout<<”Eligible for vote”;

}

Explanation

It returns true if the left value is either greater than or exactly equal to the right value.

Que19: What does <= mean?

Ans: The <= operator means **less than or equal to**.

Example:

If(age <= 18){

cout<<”Not eligible for vote”;

}

Explanation

It returns true if the left value is smaller than or equal to the right value.

Que20: What does != mean?

Ans: The != operator means **not equal to**.

Example:

If(a != b){

return a;

}

Explanation

It checks whether two values are different. If the values are not equal, the condition becomes true; otherwise, it becomes false.

Que21: What is a switch statement?

Ans: A switch statement is a decision-making statement that allows a program to execute one block of code from multiple choices based on the value of an expression.

Example:

```
switch(choice)
{
   case 1:
      cout << "Addition";
      break;
}
```

Explanation

The switch statement is useful when there are many constant choices to compare against a single variable.

Que22: What is a case label in a switch statement?

Ans: A case label represents a specific value that is compared with the switch expression.

Example:

```
case 1:
  cout << "One";
```

Explanation

When the switch expression matches a case label, the corresponding block of code is executed.

Que23: Why is break used in switch?

Ans: The break statement is used to terminate the execution of a case and exit the switch block.

Example:

```
case 1:
  cout << "One";
  break;
```

Explanation

Without break, the program continues executing the next cases even if they do not match.

Que24: What happens if break is omitted?

Ans: If break is omitted, the program executes the next case statements as well. This behavior is called fall-through.

Example:

case 1:

cout << "One";

case 2:

cout << "Two";

Explanation

After executing Case 1, the program will continue to Case 2 because there is no break statement.

Que25: What is the default case in switch?

Ans: The default case executes when none of the case labels match the switch expression.

Example:

default:

cout << "Invalid Choice";

Explanation

It acts like the else block of an if-else statement and handles unexpected values.

Que26: Can switch compare strings in C++?

Ans: No, a switch statement cannot directly compare strings in C++.

Example:

string name = "Vikram";

This cannot be used directly inside a switch statement.

Explanation

Switch statements work with integral values such as int, char, and enumerations, but not with strings

Que27: Which statement is better for many constant choices: if-else or switch?

Ans: The **switch statement** is generally better when there are many constant choices.

Explanation

A switch statement is easier to read, more organized, and often more efficient than a long if-else-if ladder.

Que28: How do you test whether a value is positive?

Ans: if(num > 0){

cout<<"value is positive";

}

Explanation

If a number is greater than zero, it is considered a positive number.

Que29: How do you test whether a value is negative?

Ans: if(num < 0){

cout<<"Value is negative";

}

Explanation

If a number is less than zero, it is considered a negative number.

Que30: How do you test whether a value is zero?

Ans: if(num == 0){

cout<<"Value is zero";

}

Explanation

If the value of a variable is exactly equal to zero, the condition becomes true and confirms that the number is zero.

Que31: Write a condition to check if age is at least 18.

Ans: if(age >= 18){

cout<,"person is adult";

}

Explanation

The condition checks whether the value of age is greater than or equal to 18. If true, the person is considered an adult and may be eligible for activities such as voting.

Que32: Write a condition to check if marks are above 50.

Ans: if(marks > 50){

cout<<"You are passed";

}

Explanation

The condition becomes true when the marks are greater than 50, indicating that the student has scored above the specified limit.

Que33: What is decision making in programming?

Ans: Decision making is the process of selecting different actions based on whether a condition is true or false.

Explanation

Programming languages use statements such as if, if-else, and switch to make decisions and control the flow of execution.

Que34: Can nested if statements have multiple levels?

Ans: Yes, nested if statements can have multiple levels.

Example:

```
if(condition1)
{
  if(condition2)
  {
    if(condition3)
    {
      // statements
    }
  }
}
```

We can use many more condition to make a nested,at least one inside condition known as a nested condition.

Explanation

One if statement can be placed inside another if statement multiple times depending on the program's requirements.

Que35: What is logical AND (&&)?

Ans: The logical AND operator (&&) combines two or more conditions and returns true only if all conditions are true.

Example:

```
if(age >= 18 && marks >= 50)
```

Explanation

If any one of the conditions is false, the entire expression becomes false.

Que36: What is logical OR (||)?

Ans: The logical OR operator (||) returns true if at least one condition is true.

Example:

if(age >= 18 || marks >= 50)

Explanation

The expression becomes false only when all conditions are false.

Que37: What is logical NOT (!)?

Ans: The logical NOT operator (!) reverses the result of a condition.

Example:

if(!status)

Explanation

If a condition is true, ! makes it false. If a condition is false, ! makes it true.

Que38: How do you check if a year is divisible by 4?

Ans: if(year % 4 == 0)

Explanation

The modulus operator (%) returns the remainder. If the remainder is 0, the year is divisible by 4.

Que39: What is a leap year?

Ans: A leap year is a year that contains 366 days instead of 365 days. February has 29 days in a leap year.

Explanation

A year is generally considered a leap year if it is divisible by 4, subject to additional century-year rules.

Que40: What is the first condition for a leap year?

Ans: year % 4 == 0

Explanation

The first condition for a leap year is that the year must be divisible by 4. If this condition is not satisfied, the year cannot be a leap year.

Que41: How do you find the largest of two numbers?

Ans: **Algorithm**

1. Start the program.
2. Input two numbers.
3. Compare both numbers using an if-else statement.
4. Display the larger number.
5. End the program.

Dry Run

Input:

a = 10
b = 20

Comparison:

10 > 20 → False

Output:

Largest Number = 20

Code:

```
#include <iostream>

using namespace std;


int main()

{

   int a, b;


   cout << "Enter two numbers: ";
```

```
    cin >> a >> b;


    if(a > b)
        cout << "Largest Number = " << a;
    else
        cout << "Largest Number = " << b;


    return 0;
}
```

Output Screenshot:

Enter two numbers: 10

20

Largest number = 20

Logic Explanation

The program compares two numbers using the greater-than (>) operator. The larger number is displayed as output.

Que42: How do you find the largest of three numbers?

Ans: **Algorithm**

1. Start the program.
2. Input three numbers.
3. Compare all three numbers using if-else-if statements.
4. Display the largest number.
5. End the program.

Dry Run

Input:

a = 10
b = 25
c = 15

Comparison:

25 > 10 and 25 > 15

Output:

Largest Number = 25

Code

```
#include <iostream>
using namespace std;

int main()
{
  int a, b, c;

  cout << "Enter three numbers: ";
  cin >> a >> b >> c;

  if(a >= b && a >= c)
    cout << "Largest Number = " << a;
  else if(b >= a && b >= c)
    cout << "Largest Number = " << b;
  else
    cout << "Largest Number = " << c;

  return 0;
}
```

Output Screenshot:

Enter three numbers: 10

15

25

Largest Number = 25

Logic Explanation

The program compares all three numbers and displays the greatest value using an if-else-if ladder.

Que43: What is an if-else-if ladder?

Ans: **Algorithm**

1. Start.
2. Check the first condition.
3. If false, check the next condition.
4. Continue until a condition becomes true.
5. Execute the corresponding block.
6. Stop.

Dry Run

Example:

Marks = 83

Conditions:

Marks >= 90 → False
Marks >= 83 → True

Output:

Grade B

Code

#include <iostream>

using namespace std;

int main()

{

Int marks;

Cout<<"Enter your marks:";

Cin>>marks;

```
if(marks >= 90)
   cout << "Grade A";
else if(marks >= 75)
   cout << "Grade B";
```

```
else
   cout << "Grade C";

return 0;

}
```

Output Screenshot

Enter your marks: 83

Grade B

Logic Explanation

An if-else-if ladder is used when multiple conditions need to be checked one after another.

Que44: What is the difference between if and switch?

Ans: **Algorithm**

1. Study both decision-making statements.
2. Compare their working.
3. Identify suitable use cases.

Dry Run

Example:

If → Can check ranges and complex conditions.
Switch → Checks fixed constant values.

Code

```
#include <iostream>

using namespace std;

int main()

{

// if statement
if(age >= 18)
   cout << "Adult";
```

```
// switch statement
switch(choice)
{
   case 1:
      cout << "Addition";
      break;
}

return 0;

}
```

Output Screenshot

(Not applicable.)

Logic Explanation

if	**switch**
Used for complex conditions	Used for fixed choices
Supports relational operators	Uses constant values
More flexible	Easier for menu-driven programs

Que45: Can switch use ranges directly?

Ans: **Algorithm**

1. Consider switch syntax.
2. Check whether ranges are allowed.
3. State the result.

Dry Run

Example:

case 1:
case 2:
case 3:

Valid

case 1-5:

Invalid

Code

#include <iostream>

using namespace std;

int main()

{

```
switch(num)
{
   case 1:
      cout << "One";
      break;
}
```

return 0;

}

Output Screenshot

(Not applicable.)

Logic Explanation

Switch statements cannot directly use ranges such as 1-5. Each case must contain a constant value.

Que46: What is fall-through in switch?

Ans: **Algorithm**

1. Execute a case.
2. Omit the break statement.
3. Continue executing subsequent cases.

4. Stop when a break is encountered.

Dry Run

Input:

choice = 1

Execution:

Case 1 executed
Case 2 executed

Code

```
#include<iostream>

using namespace std;

int main()

{

switch(choice)
{
   case 1:
      cout << "One";

   case 2:
      cout << "Two";
}

return 0;

}
```

Output Screenshot

One

Two

Logic Explanation

Fall-through occurs when the break statement is omitted, causing execution to continue into the next case.

Que47: What data type is usually used for conditions?

Ans: **Algorithm**

1. Start the program.
2. Evaluate a condition.
3. Store the result in a Boolean variable.
4. Display the result.
5. End the program.

Dry Run

Condition:

5 > 3

Evaluation:

5 > 3 = true

Output:

Condition is True

Code

```
#include <iostream>
using namespace std;

int main()
{
  bool result = (5 > 3);

  if(result)
    cout << "Condition is True";
  else
    cout << "Condition is False";

  return 0;
}
```

Output Screenshot

Condition is True

Logic Explanation

The bool data type is commonly used for conditions because it can store only two values: true or false. Most conditional statements such as if, if-else, and loops rely on Boolean values to determine the flow of execution.

Que48: Why are conditional statements important?

Ans: **Algorithm**

1. Start the program.
2. Check a condition.
3. If the condition is true, perform one action.
4. Otherwise, perform another action.
5. End the program.

Dry Run

Input:

Age = 20

Condition:

Age >= 18

Result:

True

Output:

Eligible to Vote

Code

```
#include <iostream>
using namespace std;

int main()
{
   int age;

   cout << "Enter Age: ";
   cin >> age;
```

```
  if(age >= 18)
    cout << "Eligible to Vote";
  else
    cout << "Not Eligible to Vote";

  return 0;
}
```

Output Screenshot

Eligible to Vote

Logic Explanation

Conditional statements are important because they allow a program to make decisions based on different situations. They help in controlling the flow of execution and enable programs to solve real-world problems effectively.

Que49: Give one real-life example of a conditional statement.

Ans: **Algorithm**

1. Start.
2. Check the traffic signal color.
3. If the signal is green, move the vehicle.
4. Otherwise, stop the vehicle.
5. End.

Dry Run

Condition:

Traffic Signal = Green

Result:

Move Vehicle

Output:

Go

Code

```
#include <iostream>
using namespace std;

int main()
{
  char signal;

  cout << "Enter Signal (G for Green, R for Red): ";
  cin >> signal;

  if(signal == 'G')
    cout << "Go";
  else
    cout << "Stop";

  return 0;
}
```

Output Screenshot

Enter Signal(G for Green, R for Red): G

Go

Logic Explanation

A traffic signal is a common real-life example of a conditional statement. If the signal is green, vehicles can move; otherwise, they must stop. This is similar to how an if-else statement works in programming.

Que50: Name four conditional structures covered.

Ans: **Algorithm**

1. Start.
2. Store the names of conditional structures.
3. Display them on the screen.
4. End.

Dry Run

Structures:

1. if
2. if-else
3. else-if ladder
4. switch

Output:

if
if-else
else-if ladder
switch

Code

```
#include <iostream>
using namespace std;

int main()
{
   cout << "1. if" << endl;
   cout << "2. if-else" << endl;
   cout << "3. else-if ladder" << endl;
   cout << "4. switch" << endl;

   return 0;
}
```

Output Screenshot

1. If
2. If-else
3. Else-if ladder
4. switch

Logic Explanation

The four major conditional structures covered in C++ are **if**, **if-else**, **else-if ladder**, and **switch statement**. These structures help programs make decisions and execute different actions based on specified conditions.